

Lantern Project Design Document

Living core design | Version 0.1 | 6 October 2026

A fixed camera dark fantasy action RPG built with three.js and Electron. Play a weathered male Lamplighter whose sword, carried lantern and patched rust brown longcoat define the silhouette. Combat is deliberate and responsive: move, aim, commit to a three hit combo, dodge out of recovery and use a short lantern flare to create space.



Figure 1. Historical Shadow Theatre environment concept. This establishes readable gothic composition, broad shapes and selective amber light. Its hooded hero and projectile are superseded by the selected Lamplighter and short cone flare. It is not a screenshot or a calibrated camera reference.

This document is the shared design baseline for the owner and the agents they assign. It brings the selected art direction, gameplay rules, production contracts and technical boundaries together so finished assets can replace proxies without rewriting the game.

The immediate goal is a small playable encounter and a repeatable art pipeline. The initial orthographic camera is locked at 35.264° elevation, 45° azimuth and a configurable 13 m vertical span. Calibration verification, final turnaround and the first motion sample remain production gates. A polished concept image does not make those gates complete.

Project owner controls all Codex agent assignments. This document gives implementation guidance; it does not authorize an assistant or agent to recruit, orchestrate, publish or deploy work.

Decisions and current scope

Read confirmed choices as requirements and provisional choices as changeable defaults

Status	Decision
Confirmed	three.js and Electron; fixed orthographic camera at 35.264° elevation and 45° azimuth, with configurable 13 m vertical span. Translation or follow allowed.
Confirmed	Male Lamplighter exactly as the locked original C / RUST crop: rust brown patched coat, original face, belts, vest, shirt, trousers, boots and props. No further hero changes. Existing Clean Ink rendering is selected.
Confirmed	WASD movement and mouse aim; sword and carried lantern; Shift directional dash with brief invulnerability; one lantern ability.
Confirmed	Deliberate responsive three hit combo with mouse directed sword arcs; dodge cancels designated recovery only; no initial stamina system.
Confirmed	Mouse aimed short cone flare with brief stagger and cooldown; travel facing locomotion and aim facing attacks and casts.
Confirmed	Fixed loadout first; small connected hand authored areas and simple elevation; room reset on death.
Confirmed	Painted environment lighting, contact shadows and selective glow; dynamic lighting optional.
Confirmed	One canonical 3D rig/source per actor, Clean Ink render proof and walk/attack tests in a few directions before mass export; layered 2D rig fallback if the look fails.
Confirmed	Windows primary target, macOS development; target 2560 × 1440 at 60 FPS on a named reference machine; 120 FPS stretch goal.

Provisional and unresolved

Eight authored headings, TypeScript, 60 Hz simulation, LMB sword and RMB flare are working defaults. Match the locked reference's anatomical right hand sword and left hand lantern in every production view. Gear stays modular in the canonical source; bake only the required combinations into runtime sprites.

The confirmed camera uses elevation 35.264389682754654°, displayed as 35.264°, azimuth 45° and initial vertical span 13 m. The 30° / 35° / 45° study is history, not the production bake. Hero scale, source canvas/density and draft timings remain illustrative.

Deferred and outside the first slice

Procedural generation, stacked floors, complex bridges, multiplayer, backend accounts, full inventory or loot systems, visible equipment swapping, torso independent aiming, dedicated backward and strafe clips, a general editor, automatic WebGPU migration and release distribution. A separate run clip is optional after the walk proof.

Navigation: gameplay 3; art 4–5; camera 6–7; scene composition 8; animation and assets 9–11; runtime and desktop 12–14; acceptance 15; prompts 16–17; change control and references 18.

Combat and the first encounter

Confirmed behavior with data driven tuning

Action	Input	Required behavior
Move	WASD	Normalized diagonal motion; body faces travel.
Aim	Mouse	Continuous world aim, separate from quantized visual facing.
Sword	LMB default	Three hit combo; mouse directed arcs; explicit link and reset windows.
Dodge	Shift	Directional dash with brief invulnerability; recovery cancel only.
Lantern	RMB default	Short aimed cone, brief stagger, cooldown.
Pause	Escape default	Intentional pause and focus handling; clear held input.

Weight without input friction

Each sword action has anticipation, an active strike and recovery. Preserve commitment during anticipation and the active strike. Allow dodge cancellation only in designated recovery windows. A small configurable input buffer and explicit combo link windows keep the three hits responsive without unrestricted canceling. Each target can be hit once per swing, rather than once per entire combo.

Attack arcs follow mouse aim. Whether heading locks at the start of each swing is a provisional tuning policy. Aim facing idle and dash facing locks are also defaults to document. Travel direction remains the locomotion facing rule so forward walk art does not slide backward when the pointer is behind the hero.

Lantern and dodge

The flare checks a bounded cone from continuous mouse aim, applies one brief stagger per affected enemy per cast and respects cooldown. Expose its cone and affected targets in debug view. Do not add a mana, stamina or status stacking system. Keep dash direction selection, travel distance, invulnerability interval and cancel timing explicit in gameplay data.

No damage values, combo durations, movement speeds, buffer lengths, dash distance, invulnerability frames, flare angle, range, stagger duration or cooldown are approved numerical values. Tune them in a visible test encounter. Visual frame count must not determine these values.

Smallest playable scope

Recommended scope is two tiny linked spaces, one simple melee enemy archetype, one shallow traversable height change and a minimal health and cooldown HUD. Demonstrate all three attacks, dodge, flare, damage, death and room reset. The precise reset contents and restart trigger remain implementation details to document; no progression penalty or resource loss is implied.

Early one room and one swing proofs are valid checkpoints. They do not replace the three hit connected area handoff. Enemy identity, encounter numbers and final environment kit remain production proposals, not approved finished content.

The locked Lamplighter reference

Original C RUST costume crop with no further hero changes



Figure 2. The exact user locked original C / RUST crop. C identifies this costume option, not a drawing style. Preserve these pixels as the design reference. No recoloring, refinement or regeneration is authorized. This is a concept reference, not a calibrated production sprite.

Preserve the original design exactly

Keep the weathered male face, tousled dark hair with gray at the temples, salt and pepper stubble, open cream shirt, dark buttoned vest, original crossing waist belts, trousers and strapped boots. Preserve the rust brown coat, lapels, visible repaired patches, seams, wear and uneven hem exactly as shown. Keep the sword, brass lantern, right wrist wrap and left forearm bracer on their pictured anatomical sides.

This original crop supersedes charcoal, crimson, two tone and later drifting hero outputs. Do not reinterpret the coat cut from another option, clean away its patches, alter the face or belt arrangement, or add a front panel. The picture takes precedence over earlier prose describing the costume.

Existing Clean Ink rendering is selected. C identifies the RUST costume, not a drawing style option. Preserve this exact identity and clean hard edged illustration; later style and theme experiments are historical only.

World art and visual readability

Handcrafted choices should remain legible at gameplay size



Figure 3. Historical Thorn and Ember environment concept sheet. Useful references for broad silhouettes, organic curves, restrained detail and selective amber accents. Individual objects, palette accents and the implied camera are exploratory, not an approved asset list or production bake.

Art direction rules

Build a broad dark fantasy world that can include occult, horror, steampunk, magic and gothic subjects, unified by the chosen visual style. No single tested theme family is mandatory. Use clear silhouettes, calm playable ground and a few deliberate shapes. Shadow Theatre contributes staged dark masses; Hades and Absolum inform action readability; Hollow Knight and Silksong inform expressive shape design. Diablo II and Path of Exile II inform dark fantasy atmosphere. These references do not authorize copying their characters or assets.

The handcrafted quality comes from purposeful shape decisions, restrained structural lines, character acting and consistent material simplification. Use two or three large value regions where they help clarity. Avoid grain, texture overlays, AI speckling, indiscriminate cracks, ornamental filigree and noisy realism. Darkness must not conceal telegraphs or enemy silhouettes.

Keep the floor quieter than the actors. Establish painted lighting and contact shadows before glow. Reserve amber for the lantern and meaningful effects; control actor and pale bone contrast. Inspect images at actual game scale on both light and dark grounds. Last Light, Ivory Vigil, Briar Glass and later drawing style experiments are historical explorations. None is an adopted theme family or replacement rendering style. The project uses the locked C / RUST design and existing Clean Ink treatment across its broad dark fantasy setting.

Exact angle camera study

Decision history before the final true isometric camera choice

The comparison uses the same actual 3D geometry, hero pose, lighting, world scale and target. Elevation changes alone. These are original art study renders, not final production character sprites and not evidence of sprite occlusion or gameplay performance.



Figure 4. Exact 30°, 35° and 45° study. The middle is 35°, not the chosen 35.264° production elevation. The study uses 9 m vertical span; the initial game uses 13 m. The historical crimson coat proxies do not establish the selected production drawing style.

Study settings: 1920 × 1080; 9 m vertical coverage and 16 m horizontal coverage. The source uses Blender coordinates and records azimuth 45° east of south. It must be mapped explicitly into the game's Y up convention. This closer art study is distinct from the confirmed initial 13 m gameplay framing. The source evidence remains unchanged.

Camera scale and aiming contract

Locked camera with configurable 13 metre gameplay framing

The initial orthographic camera is confirmed: elevation 35.264389682754654° above ground, azimuth 45° measured from +Z toward +X, and 13 m effective vertical span. Display elevation as 35.264° and retain full precision in the contract. At 16:9, horizontal coverage is 23.1111 m. Keep camera zoom 1 and express framing through the frustum. The 13 m span is configurable; no further approval is needed to implement this initial setup.

Elevation	10 m comparison	13 m initial	16 m comparison
35.264° selected	211.6 px	162.8 px	132.3 px

Table 1. Projected height of a vertical 1.8 m ruler at the selected full precision elevation and a 1440 px buffer height. The 13 m column is the initial framing. The formula is ruler height $\times \cos(\text{elevation}) \times \text{buffer height} \div \text{effective vertical span}$. The 1.8 m ruler is provisional; animated silhouette bounds also include the sword and lantern.

One shared camera and asset space

Use a right handed world, XZ ground and +Y height. Heading 0 points +Z; positive yaw turns toward +X. Record projection, exact angles, camera basis or matrix, coordinate mapping, framing, source density, image origin and asset bake version in one machine readable contract consumed by the renderer, exporter and compiler.

For azimuth a and elevation e , target to camera is $(\cos(e) \sin(a), \sin(e), \cos(e) \cos(a))$. Screen right is $(\cos(a), 0, -\sin(a))$; screen up is $(-\sin(e) \sin(a), \cos(e), -\sin(e) \cos(a))$. State matrix layout and units. Camera follow translates camera and target together. Do not orbit or auto zoom during normal play.

Viewport and aim must agree

Keep vertical coverage stable on resize; horizontal coverage follows aspect. Separate logical window size, CSS viewport, device pixel ratio, effective DPR, physical drawing buffer and render scale. With camera zoom 1, frustum top and bottom are $\pm \text{span} / 2$. Orthographic distance does not set zoom. Avoid multiplying Retina resolution twice.

Map screen relative WASD onto the horizontal camera basis and normalize diagonals. Cast pointer rays from the actual active canvas rectangle. Use the walkable surface or a documented local support plane on raised ground. Guard invalid rays, zero size windows and zero aim vectors; retain last valid aim. Cross level combat remains outside an unproven local plane test.

Changing bake orientation requires a new contract and regenerated or revalidated assets. Framing changes affect footprint and source density but do not by themselves change bake angle. Never disguise a mismatch with nonuniform sprite scaling. The selected true isometric 35.2643897° is distinct from the study's exact 35° . Do not relabel study images as production bakes.

Reference note: Blizzard's GDC 2022 presentation Resurrecting a Classic identifies the original Diablo II camera as 30° downward. This reference does not override Lantern's selected 35.264° elevation. A 26.565° screen edge slope is a different measurement. Path of Exile II's exact camera angle remains unverified.

Diablo II source: <https://www.gdcvault.com/play/1028028/Resurrecting-a-Classic-Bringing-Diablo>

How the 2D and 3D scene fits together

Composed art on a simple spatial foundation

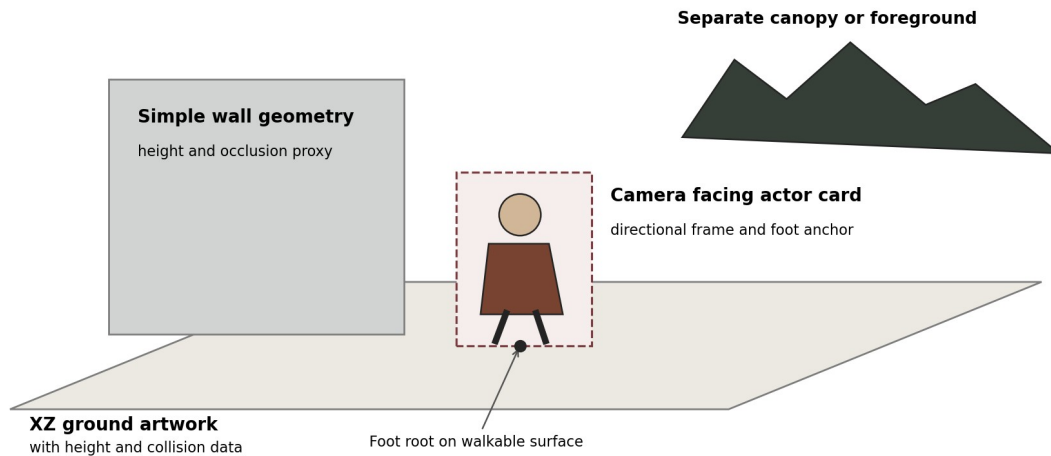


Figure 5. Schematic only. The game composes separate ground, spatial proxies, actor cards and foreground art. It does not rely on one painted background image to supply collision, depth or elevation.

Grounded actors and authored depth

Every actor has a simulation foot position on the current walkable surface. Presentation, shadows, aim indicators and debug collision derive from that root. One shared height query serves movement, aim and rendering. Start flat, then prove a shallow ramp or raised landing with one walkable height per XZ position. Painted stairs may sit above a simple ramp collision surface.

Use opaque ground and geometry; cutout actor and prop cards; soft contact shadows; separately authored foreground or canopy pieces; and translucent effects. A wall can use simple geometry with artwork on its surfaces. A tree may split its trunk/base from a foreground canopy. Collision footprints are gameplay data rather than the whole visible silhouette.

Occlusion policy

Transparent object sorting alone cannot solve tall or crossing sprites. Choose and test a coherent combination of depth tests, depth writes, alpha testing, blending and stable foot based ordering. Alpha tested cutouts are worth testing for hard edges; preserve the intended edge treatment. Keep blended effects in a suitable separate policy.

Use the least complex split layers or depth proxies that pass the scene. Check transparent holes so invisible proxies do not create solid rectangles. Avoid disabling depth tests globally or fixing individual scenes with unexplained render order values. If a bounded foreground fade or hero silhouette is needed for combat visibility, make it an explicit rule.

Required overlap fixtures

- Actor crossing another actor; hero in front of and behind a pillar.
- Hero partly behind a canopy; hero beside a wall and corner.
- A foreground element concealing the hero; foot and shadow registration on the height change.

Verify normal and debug views in the production sprite pipeline, not only the 3D proxy scene.

Animation and action timing

Three clocks with one gameplay authority

Simulation owns consequences

The proposed runtime uses a 60 Hz fixed simulation with render interpolation and a bounded catch up policy. Visual clip time and render presentation are separate clocks. Define pause, blur, suspend and resume behavior so restoring an Electron window cannot trigger a long burst of missed gameplay.

Gameplay definitions own attack wind up, active windows, recovery, combo links, hit once per target per swing, dash movement and invulnerability, flare effects and cooldown. Visual animation follows those action timelines. Adding drawings, dropping render frames or changing monitor refresh must not alter damage or movement distance.

Direction selection

Use eight proposed authored headings: d00, d45, d90, d135, d180, d225, d270 and d315. Yaw is $\text{atan2}(\text{direction.x}, \text{direction.z})$. Quantize only visual facing; retain continuous aim for gameplay. Test wraparound, ties and optional small hysteresis. Under the selected +X/+Z camera, d45 faces toward the camera and d225 away; verify with arrows in the lab.

Locomotion faces travel. Attacks and casts face aim. Heading locks at action start, aim facing idle and the precise dash facing rule are documented tuning defaults. Defer dedicated strafing, backward walking and independent torso aim. Eight forward walk views do not provide those animations.

Clip behavior and visual events

Use explicit state to clip mapping: idle, walk, optional run, attack_sword_01, attack_sword_02, attack_sword_03, dodge, hit, death and cast_lantern_flare. Map gameplay hurt to hit. Keep initial movement in the simulation and animation in place. Hold the final death pose until reset.

Each clip contains ordered frames, individual durations, loop behavior and identified visual notifies. Dispatch every crossed notify once even if a visual frame is skipped. Deduplicate by actor, action or animation instance, loop occurrence and event ID. Emit time zero once per entry; close old loops before new loop events; cancel future notifies on interruption.

Footsteps, whooshes, dust and lantern flashes are presentation events. Scrubbing the lab must not damage enemies or replay sound accidentally. If a data file contains both gameplay markers and visual notifies, label their authority and process gameplay only through the simulation.

The animation laboratory

Reuse the actual runtime animator. Provide clip and direction selection, play/pause, frame step, scrub, playback speed, canvas/trim/foot overlays, sockets and a notify log beside the gameplay timeline.

Demonstrate two actors sharing an atlas while playing independent clips. Inspect loops, planted feet, grips, coat acting and strike contact at real gameplay size.

Asset registration and compilation

Editable sources become validated runtime content

Four distinct stages

Keep reference concepts separate from editable authoring sources, reviewed import frames and generated runtime assets. Concepts never enter the game implicitly. Authoring sources retain provenance, license and a replayable export recipe. Runtime atlases, hashes, compiled manifests and contact sheets are reproducible generated output.

Registration is the foundation

Use one virtual untrimmed canvas and consistent projected pixel density per character. Image coordinates start at the top left, +x right and +y down. Record the source canvas, trim rectangle, foot anchor and padding for every frame. The anchor stays in untrimmed source coordinates. For density P and source pixel (x, y) , card offsets are $((x - \text{rootX}) / P, (\text{rootY} - y) / P)$. Reconstruct the trimmed corners in that same space, then place the anchor at the projected simulation foot.

Image height is not automatically world Y height. Runtime density is buffer height divided by effective vertical span; source density is a separate export choice. Derive source density and motion padding from the approved gameplay footprint, not the old 512×640 example. No hidden root drift or per frame rescaling.

Manifest and compiler requirements

- Stable IDs; schema/content versions; hashes; bundle membership; provenance; explicit proxy or production status; camera contract ID.
- Source dimensions, density, anchors, trim, color/alpha conventions and import recipe. Atlas pages and rectangles map back to originals.
- Ordered clip frames and durations, heading coverage, loops, visual notifies, explicit fallbacks, attachments and depth category.
- Dependencies, required/optional resources, collision footprint references and size estimates.

Reject missing files, duplicate IDs, wrong dimensions, nonfinite values, invalid bounds, nonpositive durations, incomplete required directions/clips, incompatible cameras and unapproved placeholders. Preserve exact path case and reject case only collisions across Windows and macOS. Keep paths inside approved asset roots.

Validate structure plus cross field arithmetic and real file contents; JSON Schema alone cannot prove correctness.

Atlas and alpha policy

Begin with RGBA PNG pages and a documented alpha convention. A proposed 2048 square page is only a starting size; respect actual GPU limits. Disable rotated packing initially. Use edge color dilation, extrusion and adequate gutters. Choose filtering and mip policy with real scale tests; two pixel padding is not universally mip safe. Do not force nearest filtering merely because the content is sprites.

Pack deterministically and publish atomically. Keep per actor UVs independent; never animate by changing shared texture offsets. Test dark and light grounds for halos, bleed, shimmer, clipping and trim invariance. Missing art cannot invent gameplay timing.

Art production gates and motion budget

Prove a small repeatable sample before multiplying exports

Confirmed production route: one canonical 3D rig/source for each actor, rendered into sprites from the locked camera. Prove the selected Clean Ink look, then a walk loop and sword attack in a few representative directions before mass export. Keep equipment modular in source and bake only required combinations. If the 3D route cannot achieve the look, use a controlled layered 2D rig with authored views and overlap order. Image generation supplies concepts, never independent animation frames.

Gate	Evidence before expansion
1 Design	Keep the locked original C / RUST crop unchanged. Later production views must verify silhouette, props and pictured handedness; do not reopen costume design.
2 Camera	Use locked 35.264° elevation, 45° azimuth and 13 m span. Verify calibration, hero scale and source density; version the shared bake contract.
3 First motion	Clean Ink render proof, then an in place walk and attack_sword_01 in a few representative views, with canonical source and deterministic export.
4 Eight headings	After the small proof passes, expand to all eight headings; inspect back views, switching, grips and consistent anatomy.
5 Complete kit	Add attacks 02 and 03, idle, dodge, flare, hit and death. Then one enemy and the small world kit.
6 Integration	Exact exports pass automated and visual QA in the compiler and runtime lab, with foot anchors, direction/attachment metadata, atlas, contact sheet and preview.

What a real first sample must show

Coordinated articulated motion, planted foot phases, controlled body bob, stable limb lengths and gear, clear anticipation/contact/recovery, believable lantern handling and consistent roots. Moving or wobbling one still image is not a walk cycle. Match visual strike contact to the simulation window. Check the seam, sliding, clipping, scale, outline popping and alpha edges before expanding.

While approvals are pending, implement the rig/export proof with clearly labeled proxy art and continue the runtime, lab and encounter. Do not promote a proxy to production or claim a planned pipeline is executed. Record the exact unavailable tool or source input if production cannot run.

Planning numbers remain estimates

The draft art plan budgets 98 unique drawings per direction, or 784 across eight directions, including optional run and all three sword strikes. Without run, the example is 720. The old one-view walk plus first swing estimate is 22 frames; the approved proof now covers a few representative directions before expansion. These are workload illustrations, not approved animation timings or a demand to create hundreds of frames before review.

Use authored holds and quick key poses where acting requires them. A display running at 60 FPS does not need 60 new drawings per second. A separate baked loadout would approximately multiply body clip exports; a future layered system needs shared timing, camera, registration and per view overlap data.

Runtime architecture

A small game with explicit ownership and replaceable art

Boundary	Responsibility
Electron main and preload	Privileged application work, validated persistence API and packaged resource boundary.
Renderer bootstrap	three.js scene, viewport, app lifecycle, loop and composition setup.
Input actions	Device input becomes normalized movement, aim and action commands.
Simulation	Actor state, movement, collision, ability timelines, hit tests, cooldown and death/reset.
Presentation	Interpolated transforms, directional clips, VFX, audio and HUD; no gameplay authority.
Asset runtime	Manifest loading, bundles, lifetime, atlas lookup and errors.
Offline asset tools	Validation, deterministic compilation, contact sheets and build reports.
Content and tests	Data definitions, repeatable fixtures and focused regressions.

Implementation defaults

Use strict TypeScript as the recommended language and ordinary data structures with clear modules. A single repository is sufficient. Pin current mutually compatible stable dependencies, retain a lockfile and record installed Node, Electron and three.js versions. Match three.js core and addons. The WebGLRenderer baseline is WebGL 2. Recheck current official APIs during implementation.

Avoid a React driven gameplay loop, custom engine framework, elaborate ECS, general editor, automatic WebGPU migration and speculative scale for thousands of enemies. Build the simplest supported package route. Preserve useful seams, then prove those seams with the actual small encounter.

Rendering and equipment

Color artwork is sRGB; masks and other data maps use a noncolor interpretation; output conversion occurs once. Check common swatches in import preview and runtime. The scene must read with painted lighting, contact shadows and selective glow. Optional dynamic lighting must not double light baked shading, and a PointLight will not magically illuminate an unlit sprite material.

Bake body, sword and lantern together for the first proposed visual set, keeping separate source rig objects. Attach effects through per frame and per direction metadata. Retain sockets such as hand_r_grip, hand_l_grip, lantern_light and sword_tip. A 2D source pixel socket is not inherently a physical world position; document its mapping or use an authored 3D approximation.

Do not implement general equipment composition now. Future layers need identical camera, canvas, root, timing and direction dependent front/behind ordering or masks. The lantern cannot always draw on top of the body. A fixed loadout keeps this initial pipeline controlled without closing that future option.

Loading lifetime and performance

Measure a bounded encounter and control resource growth

Manifest first loading

Separate boot/UI, shared hero and room bundles. Load asynchronously with deduplicated requests, explicit resource ownership or reference counts, progress/errors and cancellation or stale load suppression. Warm required assets before the encounter so the first sword swing does not trigger avoidable decode, upload or compilation stalls.

A room unload releases its meshes, materials, textures, render targets, listeners, timers and decoded images without disposing shared hero/UI resources. Removing an object from the scene does not release its GPU allocations. An interrupted request must not resurrect a discarded room on late completion. Keep retries safe and caches bounded.

Account for memory honestly

Report file size, decoded CPU memory, resident GPU texture estimates, render targets and temporary buffers separately. An uncompressed 2048×2048 RGBA8 base level is 16 MiB; a full mip chain adds roughly one third. PNG file compression does not change that base decoded allocation. Include occupancy waste, extra maps and copies. Renderer object counters are useful but are not a GPU byte meter.

A 256 MiB whole scene texture budget is an optional starting proposal until the target GPU and content exist. If adopted, include hero, enemies, world, UI, effects and postprocess buffers. Prefer sensible resolution, shared atlases and correct lifetime before sophisticated batching. KTX2/Basis is a later optimization after PNG works and support detection, alpha quality and offline decoder paths are verified.

One useful measurement pass

After the slice is playable, run a brief warmup and about one minute of a fixed seed representative encounter, plus a separate modest stress fixture. Record the named CPU/GPU, OS, display refresh, Electron/Chromium and three.js versions, build mode, physical buffer size, render scale, actor/effect count and resident content.

Report median and p95 frame time, visible stalls, draw calls, triangles, estimated asset bytes and repeated load/unload behavior. The 60 FPS target is approximately 16.7 ms per frame; 120 FPS is 8.3 ms.

RequestAnimationFrame intervals are not pure GPU timings. Use reliable GPU measurements if available and mark them unavailable otherwise.

Use packaged Electron for target claims. A browser development run or a macOS result cannot prove Windows 1440p performance. No reference hardware has been approved yet. Fix clear bottlenecks and repeat the same fixture; do not turn this milestone into an extensive profiling platform.

Required lifetime regression

Repeat room reset and load/unload until ordinary caches settle, then check for unexplained growth. Include cancellation during loading, immediate room replacement and a second actor using a shared atlas. An unchanging screenshot is not evidence that memory was released.

Electron security saves and packaging

A narrow desktop boundary with recoverable persistence

Keep privilege out of the game renderer

Keep Node and filesystem access in Electron main. Explicitly enable contextIsolation and sandbox, disable nodeIntegration and expose a minimal typed contextBridge API. Use a restrictive production CSP, packaged local content, blocked unexpected navigation/new windows and a bounded application protocol for packaged resources. Do not weaken production settings to make development easier.

Expose a specific method per operation, such as loadSettings, saveSettings, loadGame and saveGame. Never expose raw ipcRenderer, arbitrary channels, shell execution, unrestricted paths or IPC event objects. Validate payload schemas, allowed slot IDs, sizes and the expected sender/frame in main.

A small versioned save adapter

Use one interface for the browser development adapter and the actual Electron bridge. The browser adapter may use in memory or browser local storage, but tests must exercise the real bridge too. Settings and the slice's minimal game state use a versioned schema stored under the application userData directory.

Use bounded payloads, app owned slots, serialized writes and an OS appropriate atomic write strategy with backup/recovery. Test a simple migration, corrupt data, interrupted writes and newer unsupported versions. Never overwrite an unreadable save with defaults silently. Save format versions are separate from asset schema and camera versions.

Room reset on death is confirmed. Reset the current encounter from a declared baseline exactly once. Clear transient actions, hits, effects and cooldown state according to that baseline; increment an encounter generation ID and reject late callbacks or events from the old generation. No post reset damage or unintended respawn may leak through. No inventory loss, permanent penalty or new metagame is implied. Save only the minimal state the slice uses.

Development and packaged verification

Provide portable development, test and packaging commands for macOS and Windows using a supported Electron workflow. Keep assets available offline and preserve exact path case. Verify a packaged build on the OS that is actually available. Merely generating a Windows artifact does not establish that it launches on Windows.

The smoke test covers boot, movement/aim, all actions, pause/focus/resume, resize/high DPR, linked area transition, save/relaunch/load and clean exit. Record passed, failed and not run checks with reasons. Signing, credential dependent installers, store distribution and automatic updates remain later work; this foundation is not release certification.

Security and save acceptance

- Renderer has no unrestricted filesystem or Node surface.
- Malformed IPC payloads and unexpected senders are rejected.
- Corrupt or newer saves recover or fail gracefully without destructive defaults.
- No external navigation or arbitrary local path access appears through game content.
- Actual desktop bridge and packaged resource paths are exercised, not just browser substitutes.

Milestones and acceptance

Every milestone should leave a runnable checkpoint

Milestone	Exit evidence
A Foundation and calibration	Bootable renderer/shell, controls, versioned camera/schema fixtures and honest candidate captures. Verify the locked camera; source density and hero scale still need production validation.
B Asset and animation lab	Deterministic compiler, atlas/manifest flow, validation/contact sheets, eight way diagnostic content, independent actors and asset replacement without gameplay edits.
C Controlled motion proof	Canonical rig/source, Clean Ink render proof, walk and first sword swing in a few directions, deterministic exports, metadata and playback of those exact assets.
D Playable micro slice	Two proposed tiny connected spaces, all three combo hits, recovery dodge cancel, invulnerable dash, flare/stagger/cooldown, enemy, damage/death/reset, HUD and simple elevation.
E Desktop handoff	Save round trip, available OS packaged smoke test, focused tests, bounded performance/lifetime report and setup/authoring instructions.

Tests that protect the design

Test asset lint failures, deterministic compilation, camera mismatch, missing resources, frame bounds, trim/pivot math, heading wrap/ties and exact path case. Test fixed input replay with seeded randomness at different render cadences; state the actual determinism guarantee rather than promising universal cross platform bit identity.

Exercise all three combo hits, link/reset/buffer rules, one hit per target per swing, allowed recovery cancels and blocked anticipation/active cancels. Test dash invulnerability and interruption; flare cone boundaries, cooldown and one stagger per cast; death/reset; skipped visual frames, loop notifies, scrub safety and shared atlas independence.

Check movement, aim, foot/shadow registration and occlusion through the height change and area transition. Check stale loads, shared lifetime, repeated resets, save migration/recovery and IPC validation. Capture readable normal and debug evidence from the actual implementation.

Implementation evidence

The implementation record identifies playable behavior, run/test/package commands, verified build location, installed versions and failed or unavailable checks. Supporting evidence includes the architecture map, current camera/art contract, asset replacement walkthrough, editable animation source/export recipe, frame sequences, contact sheets and proof that those exports run. Approved, provisional and blocked art stay clearly labeled.

Implementation assignments remain separate and owner controlled. This document is a design reference, not an instruction to start or manage agents. Changes to the locked hero or rendering treatment require a new owner decision; runtime validation and production evidence do not grant redesign authority.

Locked reference and Clean Ink style

Authoritative pixels with exact source provenance and a derived reusable brief

The image is the design authority

The original standalone C / RUST crop is locked with no further hero changes. Preserve the weathered face, dark hair with gray temples, stubble, proportions, patched rust brown coat, original two crossing waist belts, dark vest and hem, cream open shirt, trousers, boots, sword, lantern and asymmetric wrist protection. Do not recolor, refine or regenerate this character.

The pictured construction takes precedence over every earlier prompt, comparison option and later generated sheet. The coat's repairs, seams and worn hem are intentional. No charcoal or crimson substitution, new front panel, changed belts, different face or softened redesign. The sword stays in anatomical right and the lantern in anatomical left, with the narrow right wrap and larger left bracer exactly as pictured.

Exact source prompt provenance

The archived `costume_variants_prompt.txt` is the exact prompt used to create the four costume options that included C / RUST. It was a reference guided edit using the earlier leftmost Disgraced Lamplighter character, not a text only generation. The standalone user crop is the final selection. The exact prompt alone cannot reproduce its pixels or guarantee the same result without the original input and generation context.

The archival prompt is reproduced verbatim in `Lantern_Recreation_Prompts_v0_1.txt`. Its unselected A, B and D options, single utility belt wording and approximate coat length are historical generation instructions; they cannot override the two belts and actual silhouette in the locked image. C means costume C, never a Soft Illustrated or other rendering preset.

Normalized reusable Clean Ink style prompt

Derived reusable wording, not the historical source prompt: create original game art in the existing clean hard edged shaded illustration style. Use selective dark ink, confident outer silhouettes and a few purposeful tapered interior lines. Describe form through clear simple angular shadow shapes, broad clean color regions and restrained material highlights. Keep detail sparse and meaningful: structural seams, intentional repairs and practical joins. Use believable weight and readable proportions, with quiet areas that keep the object recognizable at gameplay size.

No grain, paper texture, texture overlay, speckle, painterly haze, watercolor, fluid brush hybrid, photorealism or rendered 3D finish. No noisy surface detail or decorative excess. Preserve the selected object's actual design rather than treating style matching as permission to change identity, costume, palette, props or proportions. This style brief guides other authorized assets; it is not a request for another hero generation.

Production and generation limits

The crop is a concept view, not a calibrated turnaround or animation set. Later authorized production must preserve its design while validating the locked orthographic camera, density, registration and motion. Source uncertainty is explicit; unseen structure cannot become approved design through assumption. For other authorized AI concept revisions, use fresh generation rather than editing previous generated images. No overnight mass art production is authorized by this reference document.

Recreating the world and production views

Concept generation and sprite export are different steps

Environment concept prompt

Create an original clean graphic gothic courtyard for a fixed camera 2.5D dark fantasy action RPG. Frame a broad unobstructed playable stone floor with ruined pointed arches, a restrained rear gate, a few shallow steps and dark architectural silhouettes. Use large simple stone planes and a small number of meaningful slab divisions. Keep the floor visually quiet and enemy shapes easy to distinguish. Add sparse broad foliage masses with elegant tapering curves. Mature and mysterious, with charcoal, warm gray stone, restrained crimson, pale bone and small amber light accents.

Use confident selective ink edges, two or three broad flat value regions per material and clean hard shadows. Painted illumination, grounded contact shadows and contained lantern glow establish the lighting. Avoid grain, speckles, grunge, realistic material noise, dense cracks, tiny rubble, filigree, repeated hairline marks and excessive bloom. No UI or copied franchise assets. If a hero is present, use the exact locked C / RUST Lamplighter design and pictured sword/lantern sides without changing his costume. Keep ability art a short directed cone rather than an unrelated projectile.

For isolated prop exploration, request complete silhouettes on a uniform charcoal sheet with generous gutters and visible ground roots. Begin only with a floor treatment, wall/corner, pillar or canopy tree, and a chest if needed. An exploration sheet is a concept, not a promise to produce every pictured asset or proof of calibrated projection.

Production export contract template

Use this checklist with the editable rig or controlled 2D source. Fill measured values from the approved camera contract; do not ask an image model to guess them.

Field	Required value
Identity and state	Asset ID; source/version; locked original C / RUST reference; approval status; provenance/license.
Camera	Contract ID; orthographic; elevation 35.264389682754654°; azimuth 45°; initial span 13 m; axes transform and matrix convention.
Scale and output	World scale; projected source density; virtual canvas; foot root; alpha/color settings; motion padding.
Pose and direction	Clip ID; world heading d00...d315; pose/sample time; consistent anatomical gear sides; locked lighting recipe.
Timing and attachments	Explicit frame durations; visual notifies; grip, lantern and sword tip samples with declared 2D or 3D space.
Evidence	RGBA masters; deterministic atlas/manifest; hashes; native/gameplay previews; registration and runtime QA.

Exact camera studies should render the same actual geometry, pose, dimensions, lighting and target at each angle. Preserve source files and metadata. Do not compare unrelated generated paintings, rotate old baked cards, secretly rescale the hero or label approximate views as calibrated exports.

Living document and reference register

Change decisions once and propagate their consequences

Ownership and update protocol

This is the core human readable design; the runtime, compiler and export tools consume one versioned camera/art contract. Keep the document and machine readable contracts aligned. A user decision supersedes older draft defaults. Record its date, source, status, rationale, affected files and approval owner. An implementation suggestion, concept image or passing test is not product approval.

For each change, update the original requirement and decision ledger rather than appending a conflicting rule. Bump document and relevant contract versions. A camera/bake change lists which exports must be rebuilt; a clip change lists timing and gameplay compatibility. Preserve earlier evidence. Record selected elevation 35.264389682754654°, azimuth 45° and initial span 13 m. Separate historical preview presets from these approved values.

Version 0.1 records the locked original C / RUST hero, existing Clean Ink rendering and confirmed combat. The initial camera is 35.264389682754654° elevation, 45° azimuth and configurable 13 m span. Later hero outputs, fluid brush hybrids and theme families are historical only. Production turnaround, animation and runtime validation remain separate from design approval. This reference document does not authorize mass asset production.

Source package

Primary design sources: Lantern_Foundation_Agent_Prompt.txt; Art_Production_Plan.txt; Camera_Calibration_Agent_Prompt.txt; art_contract.provisional.json; asset_manifest.example.json and asset_manifest.schema.json. The example manifest contains invented demonstration rectangles and timings, not measured production assets. The exact original C / RUST crop supplied by the owner supersedes every later generated hero concept and its prompts. Historical world images remain direction references.

Official implementation references

These official URLs are carried forward from the foundation reference check on 6 October 2026. Recheck documentation against the versions installed during implementation. Project camera choices, budgets and schemas are design decisions, not requirements from these sources.

three.js renderer and camera: <https://threejs.org/docs/pages/WebGLRenderer.html>
<https://threejs.org/docs/pages/OrthographicCamera.html>

three.js ray construction and color: <https://threejs.org/docs/pages/Raycaster.html>
<https://threejs.org/manual/pages/color-management.html>

three.js transparency and resource lifetime: <https://threejs.org/manual/pages/transparency.html>
<https://threejs.org/manual/pages/cleanup.html>

Electron security and isolation: <https://www.electronjs.org/docs/latest/tutorial/security>
<https://www.electronjs.org/docs/latest/tutorial/context-isolation>

Electron IPC and persistence paths: <https://www.electronjs.org/docs/latest/tutorial/ipc>
<https://www.electronjs.org/docs/latest/api/app>

Electron packaging and testing: <https://www.electronjs.org/docs/latest/tutorial/application-distribution>
<https://www.electronjs.org/docs/latest/tutorial/automated-testing>

Blender repeatable animation export: <https://docs.blender.org/manual/en/latest/render/output/animation.html>
<https://docs.blender.org/api/current/>